

P: como posso fazer um webproxy com python, usando o flask?

A: Você pode criar um web proxy simples com Python + Flask usando a biblioteca

requests.

A ideia é:

1. O usuário acessa seu servidor Flask.
2. O Flask faz a requisição para outro site.
3. O Flask devolve a resposta para o navegador.

Exemplo básico:

```
from flask import Flask, request, Response
import requests

app = Flask(__name__)

@app.route('/proxy')
def proxy():
    url = request.args.get('url')

    if not url:
        return "Informe uma URL", 400

    try:
        resposta = requests.get(url)

        return Response(
            resposta.content,
            status=resposta.status_code,
            content_type=resposta.headers.get('Content-Type')
        )

    except requests.exceptions.RequestException as e:
        return f"Erro: {e}", 500

if __name__ == '__main__':
    app.run(debug=True)
```

Instale as dependências:

```
pip install flask requests
```

Execute:

```
python app.py
```

Depois acesse:

```
http://127.0.0.1:5000/proxy?url=https://example.com
```

Melhorando o proxy

Você pode adicionar:

Headers do navegador

Alguns sites bloqueiam requisições sem User-Agent.

```
headers = {  
    "User-Agent": request.headers.get("User-Agent")  
}  
  
resposta = requests.get(url, headers=headers)
```

Suporte a POST

```
requests.post(url, data=request.form)
```

Proxy reverso completo

Se quiser algo parecido com Nginx:

```
@app.route('/', defaults={'path': ''})  
@app.route('/<path:path>')  
def catch_all(path):  
    destino = f"https://site.com/{path}"  
  
    resposta = requests.get(destino)  
  
    return Response(  
        resposta.content,  
        resposta.status_code,  
        resposta.headers.items()  
    )
```

Problemas comuns

CORS

Navegadores podem bloquear requisições cross-origin.

Você pode usar:

```
pip install flask-cors  
from flask_cors import CORS  
CORS(app)
```

HTTPS

Para desenvolvimento:

```
app.run(ssl_context='adhoc')
```

Segurança (importante)

Nunca deixe um proxy totalmente aberto na internet.

Riscos:

- abuso/spam
- esconder ataques
- mineração
- consumo de banda

Você deveria:

- limitar domínios permitidos
- adicionar autenticação
- aplicar rate limit
- registrar logs

Exemplo simples:

```
ALLOWED = ["example.com", "api.github.com"]

if not any(site in url for site in ALLOWED):
    return "Domínio não permitido", 403
```

Ideias de projetos com isso

- navegador minimalista
 - bypass de CORS para APIs
 - cache de páginas
 - web scraping centralizado
 - proxy para jogos web
 - túnel HTTP
 - visualizador anônimo de sites
-

Documentações úteis:

- [Flask](#)
- [Requests](#)
- [Flask-CORS](#)